Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ДОМАШНЯЯ РАБОТА

«Разработка консольного приложения с применением ООП»

ДИСЦИПЛИНА: «Архитектура автоматизированных систем обработки информации и управления»

Выполнил: студент гр. ИУ5-24Б _____ (Пуговкин М.Г.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель работы

1. Разработать классы-модели Project и Task для представления таблиц базы данных с использованием свойств, валидации данных и XML-комментариев.

2. Реализовать класс DatabaseManager для инкапсуляции всех операций с SQLite: создание таблиц, импорт данных из CSV-файлов, CRUD-операции (добавление, чтение, обновление, удаление записей).

3. Создать консольное меню для интерактивного управления данными с обработкой ошибок ввода и возможностью выбора внешнего ключа при добавлении записи.

4. Разработать класс ReportBuilder с использованием паттерна Fluent Interface для формирования отчётов через цепочку вызовов методов.

5. Реализовать три отчёта:

   – полный список задач с названиями проектов;

   – количество задач по проектам;

   – среднее значение трудоёмкости по проектам;

6. Подготовить метаграфовое ситуационное описание: схему модели данных и схему работы класса ReportBuilder.

7. Протестировать работоспособность программы и подготовить отчёт с текстом программы и результатами выполнения.

Ход работы

1. Предметная область и вариант

Предметная область варианта 20: проекты и задачи. Справочная таблица хранит проекты, основная таблица хранит задачи, связанные с проектами по внешнему ключу. Числовое поле основной таблицы — hours, трудоемкость задачи в человеко-часах

2. Структура базы данных:

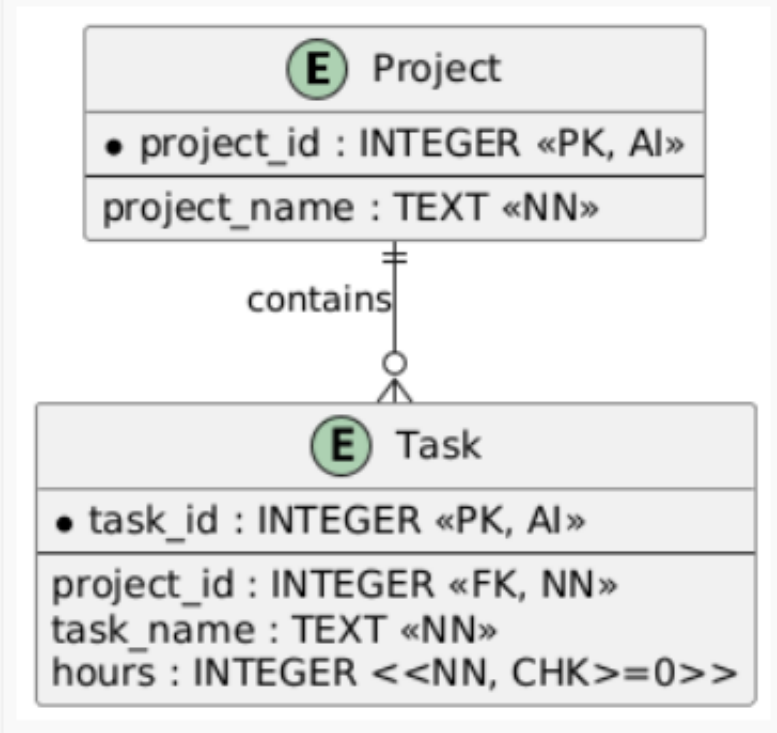

Рисунок 1 – Структура базы данных

Для предметной области был создан следующий набор данных:

Таблица 1 - данные справочника проектов

project_id	project_name
11	Backend Service
2	Mobile App
3	QA Framework
4	DevOps Pipeline

Таблица 2 - тестовые данные задач

task_id	project_id	task_name	hours
1	1	Анализ требований	30
2	1	Интеграция с БД	40
3	1	Документация	15
4	2	Дизайн интерфейса	28
5	2	Адаптация API	45
6	3	Написание тестов	22
7	4	Деплой на прод	12

8	1	Оптимизация запросов	18
9	2	Тестирование UI	35
10	3	Автотесты API	25
11	4	Мониторинг	8
12	3	Отчётность	10

3. Классы-модели

Класс Project представляет справочную сущность. Он содержит идентификатор проекта, название, конструктор с параметрами, конструктор по умолчанию и переопределение ToString() для удобного вывода объекта в консоль.

```
5   class Project
6   {
7       /// <summary>Идентификатор проекта (первичный ключ)</summary>
        Ссылок: 3
8       public int Id { get; set; }
9
10      /// <summary>Название проекта</summary>
        Ссылок: 3
11      public string Name { get; set; }
12
13      /// <summary>
14      /// Конструктор с параметрами.
15      /// </summary>
16      /// <param name="id">Идентификатор проекта</param>
17      /// <param name="name">Название проекта</param>
        Ссылок: 2
18      public Project(int id, string name)
19      {
20          Id = id;
21          Name = name;
22      }
23
24      /// <summary>
25      /// Конструктор по умолчанию (вызывает полный через this).
26      /// </summary>
        Ссылок: 0
27      public Project() : this(0, "") { }
28
29      /// <summary>
30      /// Возвращает строковое представление проекта.
31      /// </summary>
32      /// <returns>Строка формата "[Id] Name"</returns>
        Ссылок: 0
33      public override string ToString() => $"[{Id}] {Name}";
34  }
```

Рисунок 2 – Класс Project

Класс Task представляет основную сущность. Помимо идентификатора задачи и названия, класс содержит внешний ключ ProjectId и числовое поле Hours. В

свойстве Hours реализована проверка: трудоемкость не может быть отрицательной

```
Ссылок: 14
class Task
{
    /// <summary>Идентификатор задачи (первичный ключ)</summary>
    Ссылок: 4
    public int Id { get; set; }

    /// <summary>Идентификатор проекта (внешний ключ)</summary>
    Ссылок: 7
    public int ProjectId { get; set; }

    /// <summary>Название задачи</summary>
    Ссылок: 8
    public string Name { get; set; }

    private int _hours;

    Ссылок: 7
    public int Hours
    {
        get => _hours;
        set
        {
            if (value < 0)
                throw new ArgumentException("Трудоёмкость не может быть отрицательной");
            _hours = value;
        }
    }


    Ссылок: 5
    public Task(int id, int projectId, string name, int hours)
    {
        Id = id;
        ProjectId = projectId;
        Name = name;
        Hours = hours; // валидация сработает здесь
    }

    Ссылок: 0
    public Task() : this(0, 0, "", 0) { }

    Ссылок: 0
    public override string ToString() =>
        $"[{Id}] {Name}, Проект#{ProjectId}, часов: {Hours}";
}
```

Рисунок 3 – Класс Task

Класс DatabaseManager инкапсулирует работу с SQLite: создание таблиц, импорт данных из CSV, выполнение CRUD-операций, произвольных SELECT запросов для отчетов, экспорт данных и выборку задач по проекту. Это отделяет логику хранения данных от пользовательского интерфейса.

```
private void CreateTables()
{
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = @"
        CREATE TABLE IF NOT EXISTS project(
            project_id INTEGER PRIMARY KEY AUTOINCREMENT,
            project_name TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS task(
            task_id INTEGER PRIMARY KEY AUTOINCREMENT,
            project_id INTEGER NOT NULL,
            task_name TEXT NOT NULL,
            hours INTEGER NOT NULL CHECK(hours >= 0),
            FOREIGN KEY(project_id) REFERENCES project(project_id)
        );";
    cmd.ExecuteNonQuery();
}
```

Рисунок 4 - создание таблиц project и task в SQLite

Для работы с записями реализованы методы получения всех проектов и задач, поиск задачи по идентификатору, добавление, редактирование и удаление задачи. В SQL-запросах используются параметры, что повышает надежность обработки пользовательского ввода.

```
160     public Task? GetTaskById(int id)
161     {
162         using var conn = new SqliteConnection(_connectionString);
163         conn.Open();
164         var cmd = conn.CreateCommand();
165         cmd.CommandText = "SELECT task_id, project_id, task_name, hours FROM task WHERE task_id=@id";
166         cmd.Parameters.AddWithValue("@id", id);
167         using var reader = cmd.ExecuteReader();
168         if (reader.Read())
169         {
170             return new Task(
171                 reader.GetInt32(0),
172                 reader.GetInt32(1),
173                 reader.GetString(2),
174                 reader.GetInt32(3));
175         }
176         return null;
177     }
178
179     /// <summary>
180     /// Добавить новую задачу (ID генерируется автоматически)
181     /// </summary>
182     /// <param name="task">Задача для добавления</param>
        Ссылок: 1
183     public void AddTask(Task task)
184     {
185         using var conn = new SqliteConnection(_connectionString);
186         conn.Open();
187         var cmd = conn.CreateCommand();
188         cmd.CommandText = @"
189             INSERT INTO task(project_id, task_name, hours)
190             VALUES(@projectId, @name, @hours)";
191         cmd.Parameters.AddWithValue("@projectId", task.ProjectId);
192         cmd.Parameters.AddWithValue("@name", task.Name);
193         cmd.Parameters.AddWithValue("@hours", task.Hours);
194         cmd.ExecuteNonQuery();
195     }
```

Рисунок 5 - фрагменты CRUD-операций в DatabaseManager

Класс ReportBuilder формирует текстовые отчеты поверх SQL-запросов. Промежуточные методы Query(), Title(), Header(), ColumnWidths(), Numbered() и Footer() возвращают this, поэтому настройки отчета можно задавать цепочкой вызовов. Терминальные методы Build(), Print() и SaveToFile() выполняют запрос и выводят или сохраняют результат.

```
12  class ReportBuilder
13  {
14      // Жёсткая зависимость от DatabaseManager
15      private DatabaseManager _db;
16
17      // Настройки отчёта (заполняются промежуточными методами)
18      private string _sql = "";
19      private string _title = "";
20      private string[] _headers = Array.Empty<string>();
21      private int[] _widths = Array.Empty<int>();
22      private bool _numbered = false;      // [ГРУППА A]
23      private string _footer = "";         // [ГРУППА B]
24
25      /// <summary>
26      /// Конструктор. Принимает DatabaseManager для выполнения запросов.
27      /// </summary>
28      /// <param name="db">Экземпляр DatabaseManager</param>
        Ссылок: 3
29      public ReportBuilder(DatabaseManager db)
30      {
31          _db = db;
32      }
33
34      // === Промежуточные методы (возвращают this) ===
35
36      /// <summary>
37      /// Устанавливает SQL-запрос для отчёта
38      /// </summary>
39      /// <param name="sql">Текст SQL-запроса</param>
        Ссылок: 3
40      public ReportBuilder Query(string sql)
41      {
42          _sql = sql;
43          return this;
44      }
```

Рисунок 6 - реализация ReportBuilder с использованием Fluent Interface

4. Консольный клиент

В Program.cs реализован главный цикл меню. Пользователь может просмотреть все проекты и задачи, добавить задачу, отредактировать или удалить ее, перейти в подменю отчетов, выполнить фильтрацию по проекту и экспортировать данные в CSV.

```
22  // Главный цикл меню
23  string choice;
24  do
25  {
26      Console.WriteLine("\n=== Управление данными ===");
27      Console.WriteLine("1 – Показать все проекты");
28      Console.WriteLine("2 – Показать все задачи");
29      Console.WriteLine("3 – Добавить задачу");
30      Console.WriteLine("4 – Редактировать задачу");
31      Console.WriteLine("5 – Удалить задачу");
32      Console.WriteLine("6 – Отчёты");
33      Console.WriteLine("7 – Фильтр по проекту");
34      Console.WriteLine("8 – Экспорт в CSV");
35      Console.WriteLine("0 – Выход");
36      Console.Write("Ваш выбор: ");
37
38      choice = Console.ReadLine()?.Trim() ?? "";
39      Console.WriteLine();
40
41      switch (choice)
42      {
43          case "1": ShowProjects(db); break;
44          case "2": ShowTasks(db); break;
45          case "3": AddTask(db); break;
46          case "4": EditTask(db); break;
47          case "5": DeleteTask(db); break;
48          case "6": ReportsMenu(db); break;
49          case "7": FilterByProject(db); break;      // [ГРУППА Г]
50          case "8": ExportCsv(db); break;            // [ГРУППА Б]
51          case "0": break;
52          default: Console.WriteLine("Неверный пункт меню."); break;
53      }
54  } while (choice != "0");
```

Рисунок 7 - главный цикл консольного меню

5. Реализация отчетов и дополнительных функций

Приложение формирует три обязательных отчета: полный список задач с названиями проектов, количество задач по каждому проекту и среднюю трудоемкость задач по проектам. Отчеты строятся с использованием JOIN, GROUP BY, COUNT и AVG.

```
89      public ReportBuilder Footer(string label)
90      {
91          _footer = label;
92          return this;
93      }
94
95      // === Терминальные методы ===
96
97      /// <summary>
98      /// Выполняет запрос, форматирует результат через StringBuilder,
99      /// возвращает готовую строку отчёта
100     /// </summary>
101     /// <returns>Отформатированная строка отчёта</returns>
        Ссылок: 2
102     public string Build()
103     {
104         // Выполняем запрос через DatabaseManager
105         var (columns, rows) = _db.ExecuteQuery(_sql);
106         var sb = new StringBuilder();
107
108         // Заголовок отчёта
109         if (_title.Length > 0)
110         {
111             sb.AppendLine();
112             sb.AppendLine($"===={_title}===");
113         }
114
115         // Определяем названия колонок
116         string[] displayHeaders = _headers.Length > 0 ? _headers : columns;
117
118         // Определяем ширину колонок
119         int colCount = displayHeaders.Length;
120         int[] widths;
121         if (_widths.Length >= colCount)
122         {
123             widths = _widths;
124         }
125         else
126         {
127             widths = new int[colCount];
128             for (int i = 0; i < colCount; i++)
129                 widths[i] = 20;
130         }
```

Рисунок 8 - формирование отчетов

6. Демонстрация работы приложения

```
=== Управление данными ===
1 – Показать все проекты
2 – Показать все задачи
3 – Добавить задачу
4 – Редактировать задачу
5 – Удалить задачу
6 – Отчёты
7 – Фильтр по проекту
8 – Экспорт в CSV
0 – Выход
Ваш выбор: 6


--- Отчёты ---
1 – Задачи с названиями проектов (полный список)
2 – Количество задач по проектам
3 – Среднее значение часов по проектам
0 – Назад
```

Рисунок 9 - запуск приложения и переход к подменю отчетов

Первый отчет выводит задачи вместе с названиями проектов и трудоемкостью. В отчете включена нумерация строк и итоговая строка с количеством записей.

```
Ваш выбор: 1

===Задачи по проектам===
№    Задача                    Проект              Часы
-----------------------------------------------------------
1    Автотесты API             QA Framework        25
2    Адаптация API             Mobile App          45
3    Анализ требований         Backend Service     30
4    Деплой на прод            DevOps Pipeline     12
5    Дизайн интерфейса         Mobile App          28
6    Документация              Backend Service     15
7    Интеграция с БД           Backend Service     40
8    Мониторинг                DevOps Pipeline     8
9    Написание тестов          QA Framework        22
10   Настроить сервер          Backend Service     4
11   Оптимизация запросов      Backend Service     18
12   Отчётность                QA Framework        10
13   Тестирование UI           Mobile App          35
-----------------------------------------------------------
Всего записей: 13
```

Рисунок 10 - отчет «Задачи по проектам»

Второй и третий отчеты демонстрируют агрегирующие SQL-запросы: подсчет количества задач по проектам и расчет средней трудоемкости.

```
===Количество задач по проектам===
№    Проект                          Кол-во
-------------------------------------------
1    Backend Service                 5
2    DevOps Pipeline                 2
3    Mobile App                      3
4    QA Framework                    3
-------------------------------------------
Всего проектов: 4


--- Отчёты ---
1 - Задачи с названиями проектов (полный список)
2 - Количество задач по проектам
3 - Среднее значение часов по проектам
0 - Назад
Ваш выбор: 3

===Средняя трудоёмкость по проектам===
№    Проект                          Среднее часов
---------------------------------------------------
1    Mobile App                      36
2    Backend Service                 21,4
3    QA Framework                    19
4    DevOps Pipeline                 10
---------------------------------------------------
Всего проектов: 4
```

Рисунок 11 - отчеты по количеству задач и средней трудоемкости

Также проверены фильтрация задач по выбранному проекту и экспорт данных в CSV-файлы.

```
7 — Фильтр по проекту
8 — Экспорт в CSV
0 — Выход
Ваш выбор: 7

--- Фильтр по проекту ---
Доступные проекты:
  [1] Backend Service
  [2] Mobile App
  [3] QA Framework
  [4] DevOps Pipeline

Введите ID проекта: 1

Задачи проекта #1:
  [1] Анализ требований, Проект#1, часов: 30
  [3] Документация, Проект#1, часов: 15
  [2] Интеграция с БД, Проект#1, часов: 40
  [14] Настроить сервер, Проект#1, часов: 4
  [8] Оптимизация запросов, Проект#1, часов: 18

Итого: 5
```

Рисунок 12 - фильтрация по проекту и экспорт данных

Вывод:

В ходе выполнения домашней работы было разработано консольное приложение для предметной области «Проекты и задачи». Реализованы классы модели, загрузка данных из CSV в SQLite, клиентское меню для управления задачами, CRUD-операции, фильтрация, экспорт данных и генерация отчетов через ReportBuilder с применением Fluent Interface. Приложение соответствует требованиям задания: использует связь «один ко-многим» между справочной и основной таблицами, поддерживает работу с данными через консоль и формирует три обязательных отчета с агрегирующими запросами.